Projet — Effet non-linéaire TP

Travail personnel — novembre à décembre v1

Guilhem PAGES
guilhem.pages@univ-lemans.fr

Version 1
Rendu : début de la session de décembre

Problématique

Un effet audio peut présenter des non-linéarités, qui deviennent problématiques lorsqu'on traite les signaux en temps réel, et surtout de manière numérique. Nous travaillerons ici sur un effet compresseur-limiteur, largement utilisé dans le domaine musical. Un compresseur vise à limiter les passages très forts des musiques, comme les percussions ou la batterie, tout en préservant les passages plus faibles : le gain varie alors au rythme de la musique. Un limiteur, quant à lui, est très utilisé pour protéger les systèmes de diffusion (sono, enceinte, casque) : il empêche d'envoyer un signal trop fort qui abîmerait les haut-parleurs. En *off-line*, il suffirait de normaliser le signal à un niveau reproductible ; mais en temps réel, on ne connaît pas le signal qui va être diffusé ! Un compresseur est un effet *dynamique* : il ajuste le gain au niveau du signal, lentement, sans toucher à la forme d'onde. Réglé très vite avec un fort taux, le même algorithme devient un limiteur.

Objectifs:

- Coder un algorithme de compression : détecteur d'enveloppe par **moyenne récursive**, calcul et application du gain ;
- Le caractériser à l'oscilloscope : courbe statique, constantes de temps ;
- Le régler en **limiteur** et mesurer l'apparition de la distorsion : harmoniques (impaires seulement), **repliement** ;
- Proposer une solution pour éviter le repliement numérique et l'implémenter ;
- Confronter chaque mesure à la valeur attendue et l'interpréter physiquement.

Prérequis:

- TP1 à TP6 (objet `AudioStream`, conversion et saturation, mesures à l'oscilloscope) ;
- Fonction FFT (« Math ») et mode XY de l'oscilloscope ; repliement spectral (TNS).

Ⓡ **Remarque** : Ce qui est évalué

Le code tient en une vingtaine de lignes : **il ne rapporte aucun point**. Le compte rendu est évalué sur les **mesures et leur interprétation** : pour chacune, le protocole (commandes, réglages de l'oscilloscope), une capture d'écran, les valeurs relevées dans le tableau, la valeur attendue (calculée), l'écart et son explication physique. Les questions en *italique* guident l'interprétation : y répondre dans le compte rendu.

Ⓡ **Remarque** : Organisation

Travail individuel, lancé en fin de jour 2 et terminé en autonomie, en trois parties de difficulté croissante (A : compresseur, B : limiteur, C : suréchantillonnage). Compte rendu de 6 à 8 pages (PDF) à rendre au début de la session de décembre. Le code est réparti en deux projets Arduino : `jour2/Projet_compresseur` (parties A et B, calcul à F_s) et `jour2/Projet_surechantillonnage` (partie C, calcul à $K \cdot F_s$). Les deux ont les mêmes commandes.

Ⓡ **Remarque** : Montage et réglages

Comme aux TP4 à TP6 : voie 1 de l'oscilloscope sur `LINE OUT` gauche (signal traité), voie 2 sur `LINE OUT` droite (source), déclenchement sur la voie 2. **Tous les réglages se font depuis le moniteur série**, sans recompiler (115 200 bauds, fin de ligne « Nouvelle ligne ») ; chaque commande affiche les réglages en cours :

`amplitude -20`	sinus fixe, en dBFS	`seuil -20`	seuil T (dBFS)
`salves`	salves -30 / -6 dBFS	`taux 4`	taux R
`frequence 1000`	fréquence du sinus (Hz)	`attaque 5`	attaque (ms)
`musique`, `sinus`	choix de la source	`relachement 100`	relâchement (ms)
`compresseur`	-20 dBFS, 4, 5 ms, 100 ms	`reglages`	affiche les réglages
`limiteur`	-20 dBFS, 100, 0,01 ms, 0,1 ms		

1 Principe

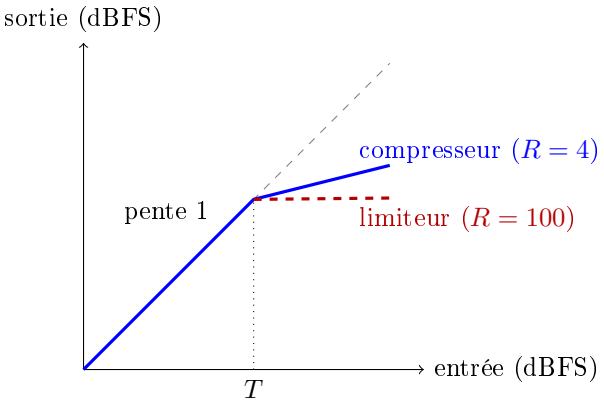
1.1 Le compresseur

Un compresseur réduit le gain quand le niveau du signal dépasse un seuil T. Au-delà du seuil, quand l'entrée monte de R dB, la sortie ne monte que de 1 dB (R : taux de compression). Il agit en trois étapes, à chaque échantillon :

1. **Détection d'enveloppe** : une moyenne *récursive* de $|x|$,

$$e[n] = a\,e[n-1] + (1-a)\,|x[n]|, \qquad a = e^{-1/(\tau f_e)},$$

   où f_e est la fréquence à laquelle on calcule.
2. **Calcul du gain** : niveau $L = 20\log_{10} e$; si $L > T$, $G = (T-L)\,(1-1/R)$ dB, sinon $G = 0$.
3. **Application du gain** : $y = x \cdot 10^{G/20}$.



> ® **Remarque** : Vers le filtrage adaptatif
>
> La moyenne récursive estime le niveau du signal en « oubliant » progressivement le passé (a est un facteur d'oubli), et le gain s'adapte à cette estimation. On retrouvera en décembre les mêmes ingrédients : estimations récursives avec facteur d'oubli (RLS), coefficients qui s'adaptent au signal à chaque échantillon (LMS).

1.2 Du compresseur au limiteur : quand le gain devient une distorsion

Un limiteur est un compresseur de taux très élevé ($R \geq 100$) : au-dessus du seuil, la sortie ne dépasse presque plus T. Pour un sinus de période T_0 :

- constantes de temps *longues* devant T_0 : le gain est presque constant sur une période ; la forme d'onde est conservée, seul le niveau change (effet dynamique) ;
- relâchement *court* devant T_0 : le gain remonte entre deux crêtes et redescend à chaque crête. Le produit $y = x \cdot g(t)$ déforme fortement la sinusoïde et crée des harmoniques ; à l'extrême, chaque crête est écrasée au niveau du seuil.

L'écrêtage du signal garanti qu'il ne dépassera pas un seuil potentiellement dangereux pour le système de diffusion (ou les oreilles), mais c'est une modification non linéaire du signal. Ces non-linéarités

ajoutent des harmoniques au signal, notamment des harmoniques impaires puisque le détecteur travaille sur $|x|$: les alternances positives et négatives sont traitées de la même façon, et le gain reprend la même forme à chaque demi-période. La sortie vérifie alors $y(t + T_0/2) = -y(t)$ et ne contient que des harmoniques *impaires* ($3f_0$, $5f_0$, …).

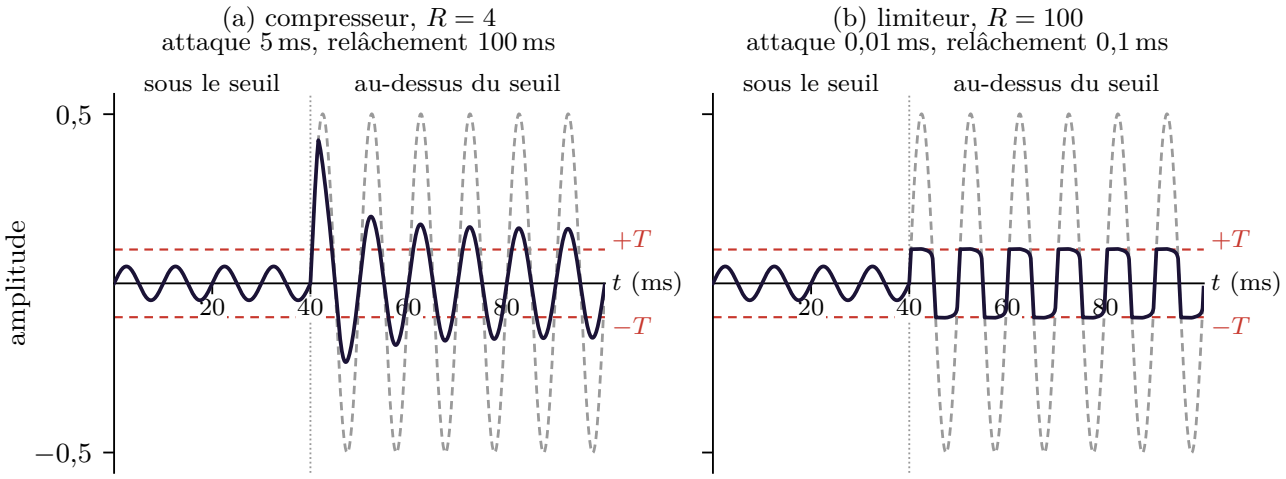
Le spectre d'un signal échantillonné à f_e est périodique de période f_e : une harmonique kf_0 apparaît à toutes les fréquences

$$f = kf_0 + m\, f_e, \qquad (k, m) \in \mathbb{Z}^2.$$

Dans la bande audio $[0, f_e/2]$, il n'en reste qu'une :

$$f_{\text{replié}} = |kf_0 - m\, f_e|\,, \quad m \text{ entier le plus proche de } kf_0/f_e.$$

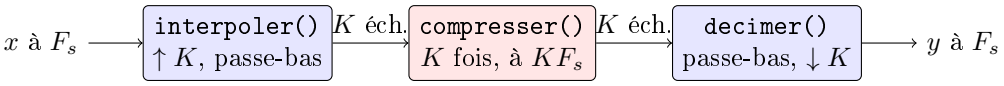
Ces raies ne sont *pas* des multiples de f_0 : on les reconnaît à l'oreille (son « métallique ») et à la FFT.



Sinus à 100 Hz, d'abord sous le seuil (−26 dBFS) puis au-dessus (−6 dBFS) ; $T = -20$ dBFS. Entrée x en tirets gris, sortie y en trait plein, simulée avec l'algorithme de `compresseur.cpp`.

1.3 Le suréchantillonnage

On fait tourner le compresseur à $f_e = K \cdot F_s$ ($K = 2$ ou 4), entre deux filtres :



- **Interpolation** : après chaque échantillon $x[n]$, on insère $K - 1$ zéros, puis on filtre par un passe-bas h qui « remplit » les trous :

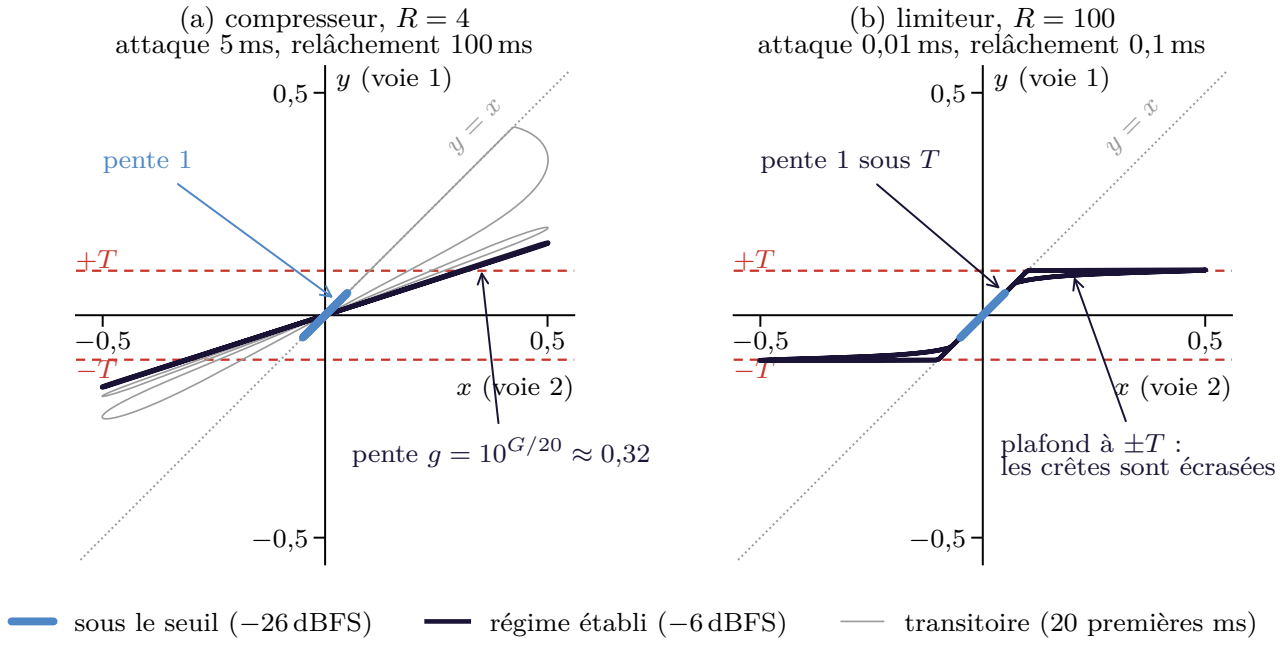
$$z[m] = \begin{cases} K\, x[m/K] & \text{si } m \text{ est multiple de } K \\ 0 & \text{sinon} \end{cases}, \qquad u[m] = \sum_{k=0}^{N-1} h[k]\, z[m-k].$$

  Le gain K compense les zéros, qui divisent la valeur moyenne par K ; le filtre supprime les $K-1$ images du spectre créées par l'insertion de zéros.
- **Décimation** : on filtre les sorties v du compresseur par le même h, puis on ne garde qu'un échantillon sur K : $y[n] = \sum_k h[k]\, v[nK - k]$.

Les harmoniques créées entre $F_s/2$ et $KF_s/2$ ne se replient plus pendant le calcul ; le filtre de décimation (coupure vers $0{,}9 \times F_s/2$) les supprime avant de revenir à F_s. Seules les harmoniques proches d'un multiple de KF_s se replient encore dans la bande audible. Le filtre a $N = 32K + 1$ coefficients (sinus cardinal fenêtré, fourni) ; les deux filtres retardent ensemble la sortie de 32 échantillons à F_s.

1.4 Mesure XY

Courbe statique. La loi entrée/sortie est une droite de pente 1 si le signal ne dépasse pas le seuil, et une droite de pente < 1 si le signal dépasse le seuil. À l'oscilloscope, il est possible d'afficher ces courbes grâce au *mode XY* : il affiche alors la sortie (voie 1, sur l'axe Y) en fonction de l'entrée (voie 2, sur l'axe X).



Mode XY de l'oscilloscope (X : voie 2, entrée x ; Y : voie 1, sortie y), mêmes signaux que la figure précédente. Une droite : seul le niveau change, la forme d'onde est conservée (effet dynamique). Une courbe : la forme d'onde est déformée (distorsion).

2 Code des parties A et B

Ouvrir `Projet_compresseur.ino`. Dans `compresseur.cpp`, compléter la fonction `compresser()`, qui traite *un* échantillon : l'enveloppe (TODO 1), le gain (TODO 2) et son application (TODO 3). Le reste est fourni : le redressement $r = |x|$ en entrée du détecteur, la boucle sur les 128 échantillons du bloc (`bloc.cpp`), les commandes du moniteur série. Faire valider la droite en XY (Q2) avant de continuer.

```
1 float AudioEffectCompresseur::compresser(float x) {
2   const float r = fabsf(x);                    // redressement : le détecteur travaille sur |x
    |
3
4   // TODO 1) enveloppe : env_ = a · env_ + (1 - a) · r,
```

```
  //         avec a = aAtt_ si r dépasse env_ (le niveau monte), aRel_ sinon

  // TODO 2) niveau = 20·log10(env_) (ajouter 1e-9f pour éviter log10(0)) ;
  //         au-dessus du seuil : gain_dB = (seuil_ - niveau) · (1 - 1/taux_), sinon 0
  float gain_dB = 0.0f;                          // provisoire

  // TODO 3) renvoyer x · 10^(gain_dB / 20)        (powf)
  return x;                                      // provisoire
}
```

3 Mesures préliminaires

Activité 1

Q0. Question préliminaire. Le signal enveloppe $e[n]$ est calculé à partir d'une équation récursive. En faire la transformée en z et calculer la fonction de transfert $H(z) = E(z)/X(z)$, en prenant $|x[n]|$ comme entrée. Quel type de filtre obtenez-vous ? Où est son pôle ? Montrer que la réponse à un échelon parcourt 63 % du chemin en τ secondes, et calculer a pour $\tau = 5$ ms et $\tau = 100$ ms à $F_s = 44{,}1$ kHz. *Pourquoi le compresseur utilise-t-il deux valeurs de a ? Le détecteur est-il encore un filtre linéaire ?*

Q1. Étalonnage et plancher de mesure. Commandes `compresseur`, `frequence 1000`, `amplitude 0`. Mesurer la tension crête de la voie 2 : c'est 0 dBFS ; en déduire la formule de conversion volts → dBFS. Puis `frequence 5000`, `amplitude -6` : FFT de la voie 2 de 0 à 25 kHz, fenêtre Flat Top ou Hanning, moyennage si disponible. Relever le niveau du fondamental, celui du plancher de bruit et des raies éventuelles, en dB par rapport au fondamental (dB). *Toute raie plus faible que ce plancher sera invisible dans la suite : quelle est la dynamique de votre chaîne de mesure ? Qu'est-ce qui la limite (convertisseur de l'oscilloscope, calibre vertical, codec) ?*

Q2. Validation du code en XY. Passer l'oscilloscope en mode XY (X : voie 2, Y : voie 1), mêmes calibres sur les deux voies. Commandes `compresseur`, `frequence 1000`, puis `amplitude -30` (sous le seuil) et `amplitude -6` (au-dessus du seuil). Capturer les deux figures et mesurer la pente de la droite dans chaque cas. *Retrouve-t-on une droite de pente 1 sous le seuil ? Au-dessus, la pente est-elle bien inférieure à 1, et la figure reste-t-elle une droite ? Que signifierait une courbe, ou une boucle qui ne se referme pas ?*

4 Partie A — Le compresseur : un effet dynamique

Réglé en compresseur ($R = 4$, attaque 5 ms, relâchement 100 ms), l'algorithme réduit l'écart entre les passages forts et les passages faibles sans déformer le signal. On le caractérise comme tout système : d'abord en régime établi (courbe statique), puis en régime transitoire (réponse à un saut de niveau).

Activité 2

Q3. Courbe statique. `compresseur`, `frequence 1000`. Pour `amplitude -40`, `-35`… jusqu'à `0`, relever l'amplitude crête de la voie 1 et la convertir en dBFS (formule de Q1) ; tracer la sortie en fonction de l'entrée, en dBFS. Refaire quelques points avec `taux 100`. *Mesurer la pente au-dessus du coude : retrouve-t-on $1/R$? Le coude tombe-t-il exactement à $T = -20$ dBFS ?*

Le détecteur mesure-t-il la crête ou une moyenne ? Comparer à la pente mesurée en XY en Q2.

Q4. Attaque et relâchement. `compresseur`, `salves` (le niveau alterne entre -30 et -6 dBFS toutes les 300 ms). Base de temps 50 ms/div, puis 2 et 20 ms/div pour les détails. Capturer une montée et une descente ; mesurer le dépassement au début de la salve forte, et le temps pour parcourir 63 % du chemin, à la montée et à la descente. Recommencer avec `relachement 30`. *Comparer aux constantes de temps réglées et au résultat de Q0. Pourquoi la sortie dépasse-t-elle au début de chaque salve forte ? Pourquoi descend-elle sous -30 dBFS juste après ?* Écouter l'effet sur la musique (`musique`).

entrée (dBFS)	-40	-35	-30	-25	-20	-15	-10	-5	0
sortie, $R = 4$ (dBFS)									
sortie, $R = 100$ (dBFS)									

5 Partie B — Le limiteur : l'apparition de la distorsion

Pour protéger un haut-parleur, on veut que la sortie ne dépasse *jamais* le seuil : on augmente le taux ($R = 100$) et on raccourcit les constantes de temps pour que le gain réagisse dès la première crête. Le prix à payer : le gain varie maintenant *à l'intérieur* d'une période, et le signal est déformé. On mesure cette distorsion, puis on cherche d'où viennent les raies qui ne sont pas des harmoniques.

Activité 3

Q5. Distorsion et relâchement. `taux 100`, `attaque 0.1`, `frequence 100`, `amplitude -6`. Pour `relachement` 100, 30, 10, 3, 1 et 0,3 ms : capturer la forme d'onde et relever, à la FFT, H2, H3 et H5 en dB. Tracer H3 en fonction du relâchement (échelle logarithmique). Puis, en mode XY, comparer `relachement 100` et `limiteur`. *À partir de quel relâchement la distorsion devient-elle nette ? Comparer à la période du sinus. Pourquoi H2 reste-t-il invisible, quel que soit le réglage ? Comment la figure XY traduit-elle la distorsion ?* Écouter les deux réglages sur la musique (`musique`).

Q6. Repliement. `frequence 5000`, `amplitude -6`, FFT de 0 à 22 kHz. Relever les raies au-dessus du plancher mesuré en Q1, d'abord avec `compresseur`, puis avec `limiteur`. Compléter la ligne « attendu » du tableau : fréquence de chaque harmonique impaire 3 à 21 après repliement ($F_s = 44{,}1$ kHz). Identifier chaque raie mesurée. *Lesquelles ne sont pas des multiples de 5 kHz ? Pourquoi le compresseur lent n'en produit-il pas ?*

relâchement (ms)	100	30	10	3	1	0,3
H2 (dB)						
H3 (dB)						
H5 (dB)						

harmonique k	3	5	7	9	11	13	15	17	19	21
kf_0 (kHz)	15	25	35	45	55	65	75	85	95	105
attendu après repliement (kHz)										
mesuré à F_s (dB)										

6 Partie C — Le suréchantillonnage : choisir, coder, vérifier

En Q6, les raies repliées du limiteur sont nettement au-dessus du plancher. Elles ne viennent pas de l'effet lui-même, mais du fait qu'on le calcule à F_s : un limiteur analogique produirait les mêmes harmoniques, mais celles au-delà de 22 kHz resteraient inaudibles. On va donc calculer le compresseur à $K \cdot F_s$, dans un second projet, `Projet_surechantillonnage` : c'est à vous de choisir $K = 2$ ou $K = 4$, **avant** de mesurer, puis de vérifier si votre choix était le bon.

6.1 Choisir K (calcul, sans carte)

Activité 4

Q7. Ce que l'on gagne. Avec un calcul à KF_s, une harmonique ne revient dans la bande 0–22 kHz que si elle tombe à moins de 22 kHz d'un multiple de KF_s. Pour $K = 2$ puis $K = 4$, calculer la plage de fréquences d'harmoniques concernées, puis les harmoniques du limiteur à 5 kHz qui se replient encore. *À l'aide de vos mesures de Q6, prévoir leur niveau : autour de quel multiple de F_s se repliaient-elles déjà pour $K = 1$? Seront-elles au-dessus du plancher mesuré en Q1 ?*

Q8. Ce que l'on paie. Par échantillon à F_s, compter les multiplications du filtre d'interpolation (K sorties de $N = 32K + 1$ produits) et de décimation (une sortie de N produits), et les appels à `compresser()`. Comparer au budget de 13 600 cycles par échantillon. Le retard ajouté vaut 32 échantillons à F_s pour $K = 2$ comme pour $K = 4$: *pourquoi ?*

Q9. Choix. Choisir K et écrire la justification (cinq lignes au plus) dans le compte rendu *avant* de passer aux mesures : on la confrontera aux résultats de Q13. Reporter ce choix dans `compresseur.h` du second projet (TODO 4).

	$K = 2$	$K = 4$
harmoniques encore repliées (plage, en kHz)		
raies attendues au-dessus du plancher		
multiplications par échantillon à F_s		
appels à `compresser()` par échantillon à F_s		

6.2 Coder l'interpolation et la décimation

Ouvrir `Projet_surechantillonnage.ino`. Le compresseur est le même qu'aux parties A et B : remplacer `compresseur.cpp` par le vôtre, sans rien y changer. Seule la fréquence de calcul change ; elle est prise en compte dans le calcul des coefficients a (`coefficient()`, dans `compresseur.h`). Le traitement d'un bloc est fourni :

```
for (int n = 0; n < AUDIO_BLOCK_SAMPLES; n++) {
  const float x = block->data[n] / 32768.0f;
  float u[K], v[K];
  interpoler(x, u);                          // K échantillons à K·Fs
  for (int p = 0; p < K; p++) v[p] = compresser(u[p]);
  block->data[n] = versInt16(decimer(v));    // retour à Fs
}
```

Les filtres sont des convolutions, écrites avec le même buffer circulaire qu'au TP4 ; les `OS_N` $= 32K + 1$ coefficients `h_[k]` sont fournis (`filtre_sinc.cpp`). Dans `compresseur.h`, régler `K` (TODO 4,

votre choix de Q9). Dans `surechantillonnage.cpp`, compléter `filtrer()` (TODO 5 : la convolution), `interpoler()` (TODO 6 : insérer $K \cdot x$ puis $K-1$ zéros, en filtrant à chaque fois) et `decimer()` (TODO 7 : ne calculer qu'une sortie filtrée sur K).

```
// Convolution, comme au TP4 : somme des h[k] · c[w - k], pour k = 0 .. OS_N - 1
float AudioEffectCompresseur::filtrer(const float *c, int w) {
  // TODO 5) s = somme des h_[k] · c[(w - k) & OS_MASK], pour k = 0 .. OS_N - 1 ; renvoyer s
  return c[w];                                    // provisoire : pas de filtrage
}
```

```
// x à Fs -> u[0..K-1] à K·Fs : insérer K·x puis K-1 zéros, en filtrant à chaque fois
void AudioEffectCompresseur::interpoler(float x, float *u) {
  // TODO 6) pour p = 0 .. K-1 :
  //         a. écrire dans zCirc_[zW_] : K·x pour p = 0, puis 0 pour p = 1 .. K-1
  //         b. u[p] = filtrer(zCirc_, zW_)
  //         c. avancer zW_ d'une case : zW_ = (zW_ + 1) & OS_MASK
  for (int p = 0; p < K; p++) u[p] = x;           // provisoire : x répété K fois, sans filtre
}
```

```
// v[0..K-1] à K·Fs -> un échantillon à Fs : on ne calcule qu'une sortie filtrée sur K
float AudioEffectCompresseur::decimer(const float *v) {
  // TODO 7) a. écrire v[0] dans vCirc_[vW_], calculer y = filtrer(vCirc_, vW_), avancer vW_
  //         b. pour p = 1 .. K-1 : écrire v[p] dans vCirc_[vW_] et avancer vW_ (sans filtrer)
  //         c. renvoyer y
  return v[0];                                    // provisoire : un sur K, sans filtre
}
```

6.3 Vérifier par la mesure

Activité 5

Q10. Validation et latence. `compresseur`, `frequence 1000`, `amplitude -30` : sous le seuil, le gain vaut 1. Avec `Projet_compresseur` puis `Projet_surechantillonnage`, comparer l'amplitude des deux voies et mesurer leur décalage temporel. *Une sortie K fois trop faible, du bruit ou des craquements signalent une erreur dans les TODO 5–6 : lesquelles ? Le retard est-il celui calculé en Q8 ?*

Q11. Bande passante. Même réglage, votre K : relever le rapport voie 1 / voie 2 (en dB) pour un sinus de 1, 15, 18, 19, 20 et 21 kHz. *Pourquoi la voie traitée perd-elle les fréquences proches de 20 kHz ? Que devient alors une vraie harmonique à 21 kHz (essayer un sinus de 7 kHz avec `limiteur`) ?*

Q12. Repliement. Réglages de Q6 avec `limiteur` : relever les raies pour votre K et les comparer à vos mesures de Q6 (calcul à F_s) et à vos prévisions de Q7. *Les raies qui restent sont-elles à de nouvelles fréquences ? Pourquoi H3 ne change-t-elle pas ?* Écouter la musique avec `limiteur`, avec chacun des deux projets.

Q13. Calcul, et bilan du choix. Relever la charge processeur avec `limiteur` pour chacun des deux projets ; comparer au rapport attendu en Q8. *Dans `interpoler()`, quelle fraction des multiplications porte sur un zéro inséré ? Combien de multiplications resterait-il si on ne faisait que les autres ? Votre choix de K était-il le bon ?* Si vos mesures le contredisent, vous pouvez changer `K` et refaire Q12 : expliquer alors ce qui vous a fait changer d'avis.

harmonique k	3	5	7	9	11	13	15	17	19	21
raie (kHz, voir Q6)										
prévu en Q7, votre K (dB)										
mesuré, votre K (dB)										

	à F_s	à KF_s	f (kHz)	1	15	18	19	20	21
retard (ms)			voie 1 / voie 2 (dB)						
charge (%)									

7 Compte rendu attendu

1. Les réglages utilisés ; la fonction de transfert du détecteur (Q0), la conversion volts $\leftrightarrow$ dBFS et le plancher de mesure (Q1) ;
2. Pour chaque mesure (Q1 à Q13) : protocole, capture, tableau des valeurs, valeur attendue, écart et explication physique ; les réponses aux questions en italique ;
3. Le choix de K (Q7 à Q9), écrit avant les mesures, puis confronté à Q10–Q13 ;
4. Une conclusion en trois points : à partir de quelles constantes de temps le compresseur devient-il une distorsion ? D'où viennent les raies qui ne sont pas des harmoniques ? Le suréchantillonnage vaut-il ce qu'il coûte, et avec quel K ?
5. En annexe : `compresseur.cpp` et `surechantillonnage.cpp` complétés.